PRODUCT REQUIREMENTS DOCUMENT

Wi-Fi Heatmap & Site Survey

NetMonitor 2.0 — macOS & iOS

Version 1.0 | March 2026

Author: Claude (Agent PRD) | Stakeholder: Blake Crane

Field	Value
Document Type	Product Requirements Document (PRD) / Design Specification
Feature	Wi-Fi Heatmap & Site Survey
Platforms	macOS 15.0+, iOS 18.0+
Phases	3 independent implementation phases
Reference Apps	NetSpot (macOS/Android), WiFiman (iOS, Ubiquiti)
Architecture	NetMonitorCore shared package + platform-specific UI
Target Agent	Daneel / Claude Code build agent

Executive Summary

NetMonitor 2.0 currently provides device discovery, ping, traceroute, port scanning, and speed testing across macOS and iOS. This PRD specifies a Wi-Fi Heatmap & Site Survey feature that adds spatial signal analysis, making NetMonitor a complete network diagnostic suite. The feature is divided into three independent implementation phases, each deliverable separately and each producing a usable feature on its own.

Phase Overview

Phase	Name	Platforms	Description	Reference
1	Blueprint Walk Survey	macOS (full), iOS (basic)	Import a floor plan image, walk the space tapping your location at each measurement point, generate heatmap overlays.	NetSpot Survey Mode
2	AR-Assisted Map + Survey	iOS only (LiDAR preferred)	Use ARKit to scan a room and auto-generate a floor plan, then walk and record measurement points on the generated map.	WiFiman Floorplan Mapper
3	AR Continuous Scan	iOS only (LiDAR required)	Walk through a space while ARKit + LiDAR continuously maps the environment and records Wi-Fi signal, producing a real-time heatmap as you move.	WiFiman Signal Mapper

Problem Statement

Network administrators, IT professionals, and power users frequently need to understand Wi-Fi coverage patterns across physical spaces. Dead zones, interference areas, and suboptimal access point placement cause connectivity problems that are invisible without spatial analysis. Today, users must purchase separate apps (NetSpot at $149–$899, or rely on Ubiquiti-ecosystem WiFiman) to perform site surveys. NetMonitor already has all the underlying network measurement capabilities but lacks the spatial visualization layer to connect signal data to physical locations.

Goals

1. Provide a professional-grade Wi-Fi site survey tool within NetMonitor that matches or exceeds NetSpot's core survey workflow on macOS.
2. Deliver AR-powered survey modes on iOS that rival WiFiman's LiDAR-based mapping, without requiring Ubiquiti hardware.
3. Reuse existing NetMonitorCore service protocols (WiFiInfoService, PingService, NetworkMonitorService) to collect signal metrics at each survey point.

4. Maintain the existing monorepo architecture: shared logic in NetMonitorCore, platform-specific UI in NetMonitor-macOS and NetMonitor-iOS.
5. Ship each phase independently so users get value incrementally.

Non-Goals

- **Predictive survey / AP placement planner** — NetSpot offers a planning mode that simulates coverage from virtual APs. This is out of scope for all three phases. It could be a future Phase 4.
- **Spectrum analysis** — WiFiman Wizard provides RF spectrum analysis via dedicated hardware. NetMonitor will not require external hardware.
- **UniFi/vendor integration** — WiFiman requires UniFi consoles for some features. NetMonitor's heatmap will be vendor-agnostic, working with any Wi-Fi network.
- **3D volumetric heatmaps** — The heatmap is a 2D floor-plan overlay. ARKit 3D mesh data is used for room boundary detection, not 3D signal visualization.
- **Cross-platform project sync** — Projects saved on macOS will not automatically sync to iOS or vice versa (iCloud sync is a future consideration).

Shared Architecture & Data Model

All three phases share a common data model and measurement engine that lives in NetMonitorCore. Platform-specific UI and input mechanisms (mouse clicks vs. AR tracking) live in the app targets. This section defines the shared foundation that must be built before any phase.

Package Placement

Component	Location	Notes
HeatmapSurveyModels	Packages/NetMonitorCore/Sources/NetMonitorCore/Models/Heatmap/	All shared value types
HeatmapServiceProtocol	Packages/NetMonitorCore/.../Services/ServiceProtocols.swift	Protocol additions
WiFiMeasurementEngine	Packages/NetMonitorCore/.../Services/Heatmap/	Shared measurement logic
HeatmapRenderer	Packages/NetMonitorCore/.../Services/Heatmap/	IDW interpolation + color mapping
macOS Heatmap UI	NetMonitor-macOS/Views/Heatmap/	Blueprint import, click-to-measure, heatmap overlay
iOS Heatmap UI	NetMonitor-iOS/Views/Heatmap/	Blueprint + AR modes, ViewModels
iOS AR Services	NetMonitor-iOS/Services/AR/	ARKit session management, LiDAR mesh processing

Core Data Model (NetMonitorCore)

The following types form the shared data layer. All are Sendable, Codable, and Identifiable.

SurveyProject

Top-level container for a heatmap survey session.

Property	Type	Description
id	UUID	Unique project identifier

Property	Type	Description
name	String	User-provided project name
createdAt	Date	Creation timestamp
floorPlan	FloorPlan	The map/image used as the base layer
measurementPoints	[MeasurementPoint]	All recorded Wi-Fi samples
surveyMode	SurveyMode	blueprint | arAssisted | arContinuous
metadata	SurveyMetadata	Building info, floor number, notes

FloorPlan

Represents the base map for the survey, whether imported as an image or generated from AR scanning.

Property	Type	Description
id	UUID	Unique identifier
imageData	Data	PNG image data of the floor plan
widthMeters	Double	Real-world width after calibration
heightMeters	Double	Real-world height after calibration
pixelWidth	Int	Image pixel width
pixelHeight	Int	Image pixel height
origin	FloorPlanOrigin	.imported(URL) | .arGenerated | .drawn
calibrationPoints	[CalibrationPoint]?	Two points with known real-world distance
walls	[WallSegment]?	Optional wall data from AR mesh (Phase 2/3)

MeasurementPoint

A single Wi-Fi signal measurement tied to a location on the floor plan.

Property	Type	Description
id	UUID	Unique identifier
timestamp	Date	When the measurement was taken
floorPlanX	Double	X position on floor plan (0.0–1.0 normalized)
floorPlanY	Double	Y position on floor plan (0.0–1.0 normalized)
rssi	Int	Signal strength in dBm (e.g., -30 to -90)
noiseFloor	Int?	Noise level in dBm (macOS only via CoreWLAN)
snr	Int?	Signal-to-noise ratio (computed or measured)
ssid	String?	Network name at measurement time
bssid	String?	Access point MAC address
channel	Int?	Wi-Fi channel number
frequency	Double?	Frequency in GHz (2.4 or 5)
band	WiFiBand?	Band enum: .band2_4GHz | .band5GHz | .band6GHz
linkSpeed	Int?	PHY link speed in Mbps (Tx rate)
downloadSpeed	Double?	Active download speed Mbps (if active scan enabled)

Property	Type	Description
uploadSpeed	Double?	Active upload speed Mbps (if active scan enabled)
latency	Double?	Ping latency to gateway in ms
connectedAPName	String?	Friendly name of connected AP if known

HeatmapVisualization (Enum)

Determines which metric is rendered as the color gradient. Mirrors NetSpot's 20+ visualization types, prioritized by usefulness.

Case	Data Source	Color Range	Priority
signalStrength	rssi (dBm)	Green (≥0 to -50) → Yellow (-50 to -70) → Red (-70 to -90+)	P0
signalToNoise	snr or rssi – noiseFloor	Green (>25dB) → Yellow (15–25) → Red (<15)	P0
downloadSpeed	Active scan Mbps	Green (>100) → Yellow (25–100) → Red (<25)	P1
uploadSpeed	Active scan Mbps	Same gradient as download	P1
latency	Gateway ping ms	Green (<10) → Yellow (10–50) → Red (>50)	P1
channelOverlap	Visible APs on same channel	Green (0–1) → Yellow (2–3) → Red (4+)	P2
frequencyBand	2.4 vs 5 vs 6 GHz	Categorical color map	P2
apCoverage	BSSID seen per point	Coverage area per AP	P2

WiFiMeasurementEngine (Service)

Shared actor in NetMonitorCore that collects Wi-Fi data at a given moment. Both platforms call this; it delegates to platform-specific WiFiInfoService for the actual readings.

Method	Signature	Description
takeMeasurement()	async → MeasurementPoint	Reads current Wi-Fi state (RSSI, SSID, BSSID, channel, band, link speed). On macOS also reads noise floor via CoreWLAN.
takeActiveMeasurement()	async → MeasurementPoint	Takes passive measurement + runs a quick speed test (3s download, 3s upload) and gateway ping.
startContinuousMeasurement()	async → AsyncStream<MeasurementPoint>	Emits passive measurements at configurable interval (default 500ms). Used by Phase 3 continuous scan.
stopContinuousMeasurement()	async	Cancels the continuous measurement stream.

HeatmapRenderer (Shared)

Generates the 2D color overlay image from measurement points using Inverse Distance Weighting (IDW) interpolation. This is a pure computation service with no UI dependencies, allowing it to live in NetMonitorCore.

- **Algorithm:** IDW interpolation with power parameter p=2.0 (standard for Wi-Fi heatmaps). Each pixel's value is the weighted average of all measurement points, where weight = 1/distance^p.
- **Output:** CGImage (via CoreGraphics, available on both platforms) with configurable resolution (default 200x200 grid, upscaled to floor plan dimensions).
- **Color mapping:** Configurable gradient with opacity. Default: green (strong) → yellow (moderate) → red (weak), 70% opacity over the floor plan.
- **Performance:** For 200x200 grid with 50 measurement points, computation takes <100ms on M1. Use Metal compute shader if >500 points (Phase 3).

PHASE 1

Phase 1: Blueprint Walk Survey

Reference: **NetSpot Survey Mode** — Import floor plan, walk space, tap locations, generate heatmap.
Platforms: **macOS (full-featured)**, **iOS (basic)**

1.1 User Stories

macOS User Stories

- **US-1.1:** As an IT administrator, I want to import a building floor plan image so I can map Wi-Fi coverage to specific physical locations.
- **US-1.2:** As a network engineer, I want to calibrate the floor plan scale by marking two points with a known distance so that coverage areas are measured in real units (meters/feet).
- **US-1.3:** As a surveyor, I want to click my current position on the floor plan and have the app automatically measure Wi-Fi signal at that point, showing a blue dot for coverage.
- **US-1.4:** As a surveyor, I want to see a live RSSI readout in the toolbar while surveying so I know if I'm in a strong or weak area before I tap.
- **US-1.5:** As a network engineer, I want to switch between 20+ heatmap visualization types (signal strength, SNR, download speed, channel overlap, etc.) to analyze different aspects of coverage.
- **US-1.6:** As an IT administrator, I want to export the heatmap as a PDF report with the floor plan, legend, and summary statistics so I can share findings with management.
- **US-1.7:** As a surveyor, I want to run an active scan at each point (speed test + ping) in addition to the passive measurement so I can see real throughput, not just signal strength.
- **US-1.8:** As a user, I want to save and re-open survey projects so I can compare before/after results when relocating access points.

iOS User Stories

- **US-1.9:** As a field technician with an iPhone, I want to import a floor plan from my photo library or Files app and perform a basic walk survey.
- **US-1.10:** As a mobile user, I want to tap my location on the floor plan to take a measurement, similar to the macOS experience but touch-optimized.
- **US-1.11:** As a mobile user, I want to see a heatmap overlay on my floor plan showing signal strength, even if fewer visualization types are available than macOS.

1.2 iOS Limitations (Phase 1)

iOS Wi-Fi APIs impose constraints that do not exist on macOS. The agent must account for these during implementation:

Capability	macOS (CoreWLAN)	iOS (NEHotspotNetwork)	Impact
RSSI	Yes, real-time via CWInterface.rssiValue()	Yes, via fetchCurrent() but requires precise location permission + connected network only	iOS can only measure RSSI on the currently-connected network; macOS can see all visible networks
Noise Floor	Yes, via CWInterface.noiseMeasurement()	No	SNR visualization unavailable on iOS (signal strength only)
BSSID	Yes, immediate access	Yes, but requires precise location permission	Must request location permission on iOS
Visible Networks	Yes, CWInterface.scanForNetworks()	No — Apple removed scan API in iOS 9	Cannot show channel overlap or AP coverage heatmaps on iOS
Channel / Band	Yes	Limited — no direct channel API, must infer from BSSID vendor data	Channel heatmap degraded on iOS
Link Speed (Tx Rate)	Yes, via CWInterface.transmitRate()	No direct API	Link speed visualization macOS-only
Background Scanning	Not needed (laptop stays open)	No background CL + NEHotspotNetwork access	Survey must be active/foreground on iOS

1.3 Functional Requirements

P0 — Must Have

6. **Floor plan import** — Accept PNG, JPEG, PDF, and HEIC images. macOS: NSOpenPanel file picker + drag-and-drop onto the survey view. iOS: PHPickerViewController + Files app integration via UIDocumentPickerViewController.
7. **Scale calibration** — Two-point calibration: user clicks/taps two points on the floor plan and enters the real-world distance between them. The app computes pixels-per-meter and displays a scale bar. Calibration is optional but recommended (uncalibrated plans show pixel coordinates).
8. **Walk survey workflow** — User taps/clicks their current position on the floor plan. App takes a passive Wi-Fi measurement (RSSI, SSID, BSSID, channel, band) and places a marker dot at that location. Blue translucent circles show coverage radius for each point. Recommended spacing guidance displayed (e.g., “Walk 3–5 meters between points for best results”).
9. **Live RSSI indicator** — Real-time signal strength bar/badge visible during survey. macOS: in the toolbar. iOS: floating overlay card. Updates every 1s via WiFiMeasurementEngine passive stream.
10. **Heatmap generation** — After collecting ≥3 points, generate IDW-interpolated heatmap overlay. Default visualization: signal strength (RSSI). Overlay rendered at 70% opacity

on top of floor plan image. Color gradient: green (≥0 to -50 dBm) → yellow (-50 to -70 dBm) → red (-70 to -90+ dBm).

11. **Project save/load** — Serialize SurveyProject as JSON + floor plan image to a .netmonsurvey file bundle. macOS: save to user-chosen directory. iOS: save to app Documents (accessible via Files app).
12. **Measurement point management** — Tap/click an existing point to see its data. Long-press/right-click to delete a point and re-render the heatmap. Undo support on macOS (Cmd+Z).

P1 — Nice to Have

13. **Active scan mode** — Toggle to run a 6-second speed test (3s download + 3s upload) + gateway ping at each measurement point. Uses existing SpeedTestServiceProtocol. Results populate downloadSpeed, uploadSpeed, latency fields on MeasurementPoint.
14. **Multiple visualization types** — macOS: signalStrength, signalToNoise, downloadSpeed, uploadSpeed, latency (5 types). iOS: signalStrength, downloadSpeed, latency (3 types, limited by API availability).
15. **PDF export** — Export heatmap view as a multi-page PDF report: page 1 = heatmap overlay on floor plan with legend; page 2 = summary statistics table; page 3 = per-point measurement data. macOS only in Phase 1.
16. **Draw floor plan** — macOS: simple canvas drawing tool for users without a floor plan image. Draw walls, doors, rooms with basic shapes. Similar to NetSpot's built-in layout editor.

P2 — Future Considerations

17. **Channel overlap heatmap** — macOS-only (requires CWInterface.scanForNetworks()). Shows interference from neighboring APs.
18. **AP coverage zones** — Color-coded overlay showing which AP (BSSID) serves each area. Useful for roaming analysis.
19. **Before/after comparison** — Load two survey projects side-by-side to compare coverage changes.
20. **iCloud sync** — Sync .netmonsurvey projects between macOS and iOS.

1.4 UI/UX Specification

macOS Layout

The heatmap survey is a new top-level item in the macOS sidebar navigation, below the existing Network Map entry.

- **Main view:** Split view. Left: floor plan canvas (zoomable, pannable via scroll/pinch on trackpad). Right: sidebar with survey controls and measurement list.
- **Toolbar:** Survey mode toggle (passive/active), live RSSI badge, visualization type picker (dropdown), undo/redo, export button.

- **Floor plan canvas:** NSView subclass backed by CALayer for smooth zoom/pan. Renders floor plan image as base layer, measurement points as blue dots with pulse animation, heatmap as semi-transparent overlay layer using HeatmapRenderer output.
- **Measurement sidebar:** Scrollable list of all measurement points with timestamp, RSSI, SSID. Click to highlight on canvas. Summary stats at top (point count, min/max/avg RSSI, coverage area).
- **Calibration sheet:** Modal sheet during project setup. Shows floor plan with two crosshair targets the user drags to known positions. Text field for distance input with unit picker (meters/feet).

iOS Layout

The heatmap survey is a new tab or a sub-section of the existing Tools tab, accessible via a prominent card on the dashboard.

- **Main view:** Full-screen floor plan canvas (UIScrollView with zoom). Floating bottom sheet (detent-based) shows survey controls and live RSSI.
- **Touch interaction:** Tap on floor plan to place measurement point. Pinch to zoom, two-finger drag to pan. Long-press a point to see detail popover with delete option.
- **Floating HUD:** Persistent floating card showing current RSSI, SSID, and point count. Matches existing liquid glass UI theme.
- **Visualization picker:** Bottom sheet segmented control for available heatmap types.

1.5 Acceptance Criteria

ID	Criterion	Platform
AC-1.1	User can import a PNG/JPEG/PDF floor plan and see it rendered on the canvas	Both
AC-1.2	Two-point calibration sets correct pixels-per-meter ratio (verified by scale bar)	Both
AC-1.3	Tapping/clicking the canvas creates a MeasurementPoint with valid RSSI within 2 seconds	Both
AC-1.4	Heatmap overlay updates within 500ms of adding a new point (for ≤50 points)	Both
AC-1.5	Heatmap correctly interpolates between points using IDW (no visual artifacts at edges)	Both
AC-1.6	Project save/load round-trips all data (floor plan, points, calibration) without loss	Both
AC-1.7	Live RSSI updates at 1Hz during active survey	Both
AC-1.8	macOS: all 5 P0+P1 visualization types render correctly	macOS
AC-1.9	iOS: signal strength heatmap renders with correct liquid glass theme styling	iOS
AC-1.10	Active scan produces valid download/upload speed at each point (within 10% of standalone SpeedTest)	Both
AC-1.11	macOS: PDF export produces a readable multi-page report	macOS

1.6 Technical Implementation Notes

macOS Wi-Fi Data Collection

macOS has rich Wi-Fi APIs via CoreWLAN framework. The existing WiFiInfoService on macOS should be extended (or a new HeatmapWiFiService created) that provides:

- CWInterface.rssiValue() for real-time RSSI
- CWInterface.noiseMeasurement() for noise floor (enables SNR calculation)
- CWInterface.ssid(), .bssid(), .channel(), .transmitRate() for full AP info
- CWInterface.scanForNetworks() for visible network list (enables channel overlap analysis)
- All calls are synchronous and fast (<10ms). Can poll at 1Hz safely.

iOS Wi-Fi Data Collection

iOS is more restricted. Use NEHotspotNetwork.fetchCurrent() which requires:

- Access WiFi Information entitlement (already in NetMonitor iOS entitlements)
- Precise location permission (CLLocationManager, already granted for network scanning)
- Returns: SSID, BSSID, signalStrength (RSSI as Double, -100 to 0), isSecure
- Does NOT return: noise floor, channel, visible networks, link speed
- Rate limit: Apple may throttle calls. Cache results for 1s minimum between polls.

IDW Interpolation Algorithm

The heatmap renderer uses Inverse Distance Weighting, the industry standard for Wi-Fi heatmaps:

- For each pixel (x, y) in the output grid, calculate distance to every measurement point
- Weight = 1 / distance^p where p = 2.0 (Shepard's method)
- Pixel value = sum(weight_i * value_i) / sum(weight_i)
- Clamp to valid range for the visualization type (e.g., -100 to 0 for RSSI)
- Map clamped value to color gradient using linear interpolation
- Render as CGImage with alpha channel for overlay transparency

File Format (.netmonsurvey)

A .netmonsurvey file is a directory bundle (UTI: com.netmonitor.survey) containing:

- survey.json — Serialized SurveyProject (all metadata, points, calibration)
- floorplan.png — Original floor plan image
- heatmap-cache/ — Pre-rendered heatmap images for fast reopening (optional)
- Register UTI in Info.plist for both macOS and iOS so files open in NetMonitor.

1.7 Success Metrics

Metric	Target	Measurement
Survey completion rate	>70% of started surveys have ≥10 points	Analytics event on project save
Heatmap render time	<500ms for 50 points, <2s for 200 points	Instrument / os_signpost
Measurement accuracy	RSSI within ±2 dBm of system Wi-Fi menu reading	Manual QA comparison
Project file size	<5MB for typical survey (50 points + floor plan)	File size audit

1.8 Open Questions

Question	Owner	Status
Should floor plan drawing tool be Phase 1 P1 or deferred to Phase 2?	Blake	Open
PDF export: use the existing pdf skill approach or integrate a Swift PDF library?	Engineering	Open
Should macOS heatmap use NSView + CALayer or SwiftUI Canvas?	Engineering	Open — CALayer recommended for zoom/pan performance
File association: register .netmonsurvey UTI in Xcode or XcodeGen project.yml?	Engineering	XcodeGen (per ADR-001)
Active scan duration: 6s total or user-configurable?	Blake	Open

PHASE 2

Phase 2: AR-Assisted Map Creation + Survey

Reference: **WiFiman Floorplan Mapper** — Use ARKit to scan room, generate floor plan, then walk and record measurement points.
Platform: **iOS only** (iPhone/iPad with ARKit support; LiDAR preferred but not required)

2.1 Overview

Phase 2 eliminates the need to import a floor plan by using ARKit to generate one. The user walks through a room while holding their iPhone, and the app creates a 2D floor plan from the AR session's world map and mesh data. Once the map is generated, the survey workflow from Phase 1 begins: the user walks the space again, tapping their location on the AR-generated floor plan to record measurements.

This is a two-step process: **Step A: Map the space** (AR scan), then **Step B: Survey the space** (walk and tap, identical to Phase 1 iOS). The map generation replaces the floor plan import step.

2.2 User Stories

- **US-2.1:** As a field technician without a floor plan, I want to walk through a room with my iPhone camera to automatically generate a floor plan layout.
- **US-2.2:** As a surveyor, I want to see a real-time preview of the floor plan being constructed as I scan, so I know when I've covered the full area.
- **US-2.3:** As a user, I want the app to detect walls and room boundaries from the AR mesh so the generated floor plan looks like an actual architectural layout.
- **US-2.4:** As a user, I want to be able to edit the generated floor plan (adjust walls, add labels) before starting the survey.
- **US-2.5:** As a surveyor, once the map is generated I want to proceed directly into the Phase 1 walk survey workflow without re-importing anything.
- **US-2.6:** As a user with a non-LiDAR iPhone, I want the AR scan to still work (using feature-point tracking instead of mesh), even if the generated floor plan is less precise.

2.3 Functional Requirements

P0 — Must Have

21. **AR session management** — Start an ARWorldTrackingConfiguration session with .sceneReconstruction = .mesh (LiDAR) or .meshWithClassification (if available). Fallback to feature-point-only tracking for non-LiDAR devices.

22. **Real-time mesh visualization** — While scanning, show the AR camera feed with detected surfaces highlighted. Walls shown as blue planes, floor as green plane. Use ARMeshAnchor data from LiDAR or ARPlaneAnchor for non-LiDAR.
23. **Floor plan generation** — Convert 3D AR mesh/plane data into a 2D top-down floor plan image: (a) Project all vertical ARMeshAnchor faces onto the XZ plane to detect walls. (b) Apply a height filter (0.5m to 2.5m above floor level) to capture walls, not furniture. (c) Rasterize wall segments into a 2D image at 10 pixels/meter resolution. (d) Run contour detection to clean up noise and produce clean wall lines. (e) Fill rooms with white, walls with black lines. (f) Output as FloorPlan with origin = .arGenerated and pre-calibrated widthMeters/heightMeters (calibration is automatic since AR provides real-world scale).
24. **Scan completion detection** — Track coverage percentage based on mesh density. Show a progress indicator: “Room 45% scanned.” Suggest the user scan missed areas (shown as grey/hatched zones on the live preview).
25. **Transition to survey mode** — When the user taps “Done Scanning,” render the final floor plan, present it in the Phase 1 survey view, and begin measurement mode. The AR-generated plan is pre-calibrated (no manual calibration needed since AR provides real-world coordinates).
26. **AR position tracking during survey** — After the map is generated, keep the AR session alive (but stop mesh updates) to track the user’s position. When the user taps “Measure Here,” use the AR session’s camera transform to automatically determine their position on the floor plan instead of requiring a manual tap. Fallback to manual tap if AR tracking is lost.

P1 — Nice to Have

27. **Floor plan editing** — After generation, allow the user to adjust wall positions, delete false walls (furniture edges), and add room labels. Simple touch-based editing: drag wall endpoints, tap to delete, text input for labels.
28. **Multi-room stitching** — Scan one room, move to adjacent room, continue scanning. AR session maintains spatial continuity. Floor plan expands as new rooms are scanned.
29. **Mesh classification** — On LiDAR devices, use ARMeshClassification to distinguish walls from floors, ceilings, doors, and windows. Render doors as gaps in wall lines, windows as dashed lines.

P2 — Future Considerations

30. **RoomPlan API integration** — Apple’s RoomPlan framework (iOS 16+, LiDAR required) provides structured room capture with walls, doors, windows, and furniture as typed objects. This would produce far cleaner floor plans than manual mesh processing. Evaluate whether RoomPlan’s CapturedRoom model can serve as the floor plan source directly.
31. **Export AR floor plan to Phase 1** — Allow saving the AR-generated floor plan as a standalone image that can be imported on macOS for a more detailed macOS survey.

2.4 ARKit Technical Implementation

Required Frameworks

- ARKit — World tracking, mesh reconstruction, plane detection
- RealityKit — AR view rendering, mesh visualization
- Metal — Custom mesh rendering and floor plan rasterization
- CoreImage — Contour detection and image cleanup
- Accelerate — Matrix operations for coordinate projection

AR Session Configuration

Setting	LiDAR Device	Non-LiDAR Device
Configuration	ARWorldTrackingConfiguration	ARWorldTrackingConfiguration
sceneReconstruction	.meshWithClassification	.none (unavailable)
planeDetection	[.horizontal, .vertical]	[.horizontal, .vertical]
environmentTexturing	.automatic	.automatic
frameSemantics	.sceneDepth	.none
Wall detection	ARMeshAnchor vertices projected to XZ	ARPlaneAnchor vertical planes only
Accuracy	~5cm wall positioning	~20–30cm plane positioning
Min scan time	30–60s per room	60–120s per room

Floor Plan Generation Pipeline

32. **Collect mesh data:** Accumulate all ARMeshAnchor geometry. For non-LiDAR, accumulate ARPlaneAnchor vertical planes.
33. **Height-filter vertices:** Keep only vertices between floor_level + 0.3m and floor_level + 2.5m (wall region). Discard floor, ceiling, and tall furniture.
34. **Project to 2D:** Transform 3D vertices to XZ plane (top-down view). Each vertex becomes a 2D point.
35. **Rasterize:** Render 2D points into a bitmap at 10px/m resolution. Apply Gaussian blur (sigma=2px) to connect nearby points into continuous walls.
36. **Edge detection:** Apply Canny edge detection or contour tracing to extract clean wall lines from the rasterized image.
37. **Vectorize (optional):** Convert pixel edges to line segments using Hough transform or Douglas-Peucker simplification. Produces cleaner walls.
38. **Render final image:** Draw wall lines (2px black) on white background. Add scale bar. Output as FloorPlan with real-world dimensions.

Position Tracking During Survey

After the map is generated, the AR session continues running to provide real-time position tracking. The user's current position is shown as a blue pulsing dot on the floor plan (like a "you are here" marker). When the user taps "Measure Here," the current AR camera position (in

world coordinates) is projected onto the 2D floor plan to automatically place the measurement point.

- **Coordinate transform:** AR world origin is set when scanning starts. All floor plan coordinates are in the same AR world frame. Projection: floorPlanX = (arPosition.x - mapMinX) / mapWidth, floorPlanY = (arPosition.z - mapMinZ) / mapHeight.
- **Tracking loss fallback:** If AR tracking state degrades to .limited or .notAvailable, hide the auto-position dot and show a message: “Tracking lost. Tap your position manually.” Survey continues in Phase 1 manual-tap mode.

2.5 Acceptance Criteria

ID	Criterion
AC-2.1	AR camera feed shows surface detection overlay within 5 seconds of starting scan
AC-2.2	LiDAR device: generated floor plan shows wall positions within 10cm of actual
AC-2.3	Non-LiDAR device: generated floor plan shows room boundaries within 30cm of actual
AC-2.4	Floor plan generation completes in <5 seconds after user taps Done
AC-2.5	Generated floor plan has correct real-world dimensions (no manual calibration needed)
AC-2.6	AR position tracking during survey places measurement dots within 20cm of actual position
AC-2.7	Survey workflow after map generation is identical to Phase 1 iOS flow
AC-2.8	AR session uses <500MB memory on iPhone 14 Pro for a 100m² room scan
AC-2.9	Scan works in rooms with normal indoor lighting (no special lighting required)

2.6 Success Metrics

Metric	Target	Measurement
AR map generation usage	>30% of iOS surveys use AR map instead of blueprint import	Analytics: surveyMode field
Map quality rating	>3.5/5 user rating on map accuracy (in-app feedback prompt)	In-app feedback modal
AR scan duration	Average <90 seconds for a single room	Timer from AR session start to Done tap
Position tracking accuracy	<30cm median error during survey phase	Manual QA with measured reference points

2.7 Open Questions

Question	Owner	Status
Should we use Apple’s RoomPlan API instead of manual mesh processing?	Engineering	Open — RoomPlan produces much cleaner results but is LiDAR-only

Question	Owner	Status
How to handle multi-floor buildings?	Design	Open — Propose separate survey projects per floor initially
Should AR session persist between map scan and survey, or restart?	Engineering	Open — Persist recommended for position tracking
Memory budget for AR mesh accumulation on older devices (iPhone 12)?	Engineering	Needs testing
Can we share the AR-generated floor plan image to macOS for Phase 1 macOS survey?	Blake	Open

PHASE 3

Phase 3: AR Continuous Scan (WiFiman-Style)

Reference: **WiFiman Signal Mapper** — Walk through space with LiDAR; map is drawn and colored by Wi-Fi signal in real time.
Platform: **iOS only** (**LiDAR required** — iPhone 12 Pro and later, iPad Pro 2020 and later)

3.1 Overview

Phase 3 is the most advanced mode and the closest experience to WiFiman's Signal Mapper. Instead of the two-step process of Phase 2 (scan room, then survey), Phase 3 does everything simultaneously: as the user walks through a building, the app continuously maps the space using LiDAR/ARKit AND records Wi-Fi signal strength. The floor plan and heatmap are generated in real time as a single unified experience.

The user simply points their iPhone at the ground/walls and walks. The app shows a top-down map that grows as they move, with colors painted in real time showing signal quality. **There is no separate tap-to-measure step.** The entire process is passive and continuous.

3.2 User Stories

- **US-3.1:** As a network engineer, I want to walk through an entire building floor and have my iPhone automatically map the space and record Wi-Fi signal everywhere I walk, producing a complete heatmap in one pass.
- **US-3.2:** As a user, I want to see the map and heatmap colors appearing in real time on my screen as I walk, so I can immediately see dead zones and strong areas.
- **US-3.3:** As a field technician, I want the scan to work reliably in hallways, open offices, and rooms with normal furniture without requiring special setup.
- **US-3.4:** As a user, I want to see which AP (access point) my device is connected to change as I roam, overlaid on the map.
- **US-3.5:** As a user, I want to pause scanning (e.g., to take a call), then resume from where I left off without losing data.
- **US-3.6:** As a user, once the scan is complete I want to switch between visualization types (signal strength, speed, latency) on the generated map, re-rendering the heatmap colors.

3.3 Functional Requirements

P0 — Must Have

39. **Continuous AR + Wi-Fi capture** — Run ARWorldTrackingConfiguration with .meshWithClassification (LiDAR) while simultaneously polling Wi-Fi signal via WiFiMeasurementEngine.startContinuousMeasurement() at 500ms intervals. Each measurement is tagged with the current AR position.
40. **Real-time top-down map rendering** — As the user walks, project the accumulated AR mesh onto a 2D top-down view and render it as the floor plan. The map grows incrementally — newly scanned areas appear at the edges. Use Metal for rendering performance.
41. **Real-time heatmap coloring** — As measurements come in, color the corresponding areas of the 2D map according to the signal strength gradient. Each measurement paints a circular area (radius ~1.5m) around the user's position. Colors: blue/green (strong, -30 to -50 dBm) → yellow (-50 to -70 dBm) → red (weak, -70 to -90+ dBm). Note: WiFiman uses blue for strong, red for weak. We should match this convention.
42. **Split-screen view** — Top half: live AR camera feed with floor detection overlay. Bottom half: growing 2D floor plan with heatmap colors. The user's position shown as a pulsing dot on both views.
43. **Pause/resume** — Button to pause the scan (stops AR tracking + Wi-Fi measurement). Resume relocates the AR session and continues from the last known position. Data from the pause period is not interpolated.
44. **Scan completion** — User taps "Finish Scan" to end the session. App runs final floor plan cleanup (contour smoothing, gap filling) and full IDW interpolation to produce the final polished heatmap. Saves as a SurveyProject with surveyMode = .arContinuous.
45. **Post-scan visualization switching** — After scan completes, user can switch visualization types using the Phase 1 visualization picker. The measurement points are dense enough for smooth IDW interpolation.

P1 — Nice to Have

46. **AP roaming overlay** — Show BSSID changes on the map as colored boundary lines. When the device roams from AP-A to AP-B, draw a dotted line at the transition point. Each AP's coverage area gets a distinct border color.
47. **Walking path trace** — Show the user's walking path as a thin line on the 2D map. Helps identify areas that were missed (no path = no coverage data).
48. **Coverage completeness indicator** — Show what percentage of the scanned area has Wi-Fi measurements. "85% covered — walk through the kitchen to complete."
49. **Quick re-scan areas** — Highlight areas with low measurement density or high variance. User can walk those areas again to improve data quality.

P2 — Future Considerations

50. **Multi-floor building scan** — Detect floor transitions (elevator, stairs) via barometer + AR tracking discontinuity. Auto-create new floor plan layer.
51. **WiFiman Wizard hardware support** — Pair with a spectrum analyzer via Bluetooth for richer RF data beyond what iOS Wi-Fi APIs provide.
52. **Video recording overlay** — Save the AR camera feed as a video with heatmap overlay for presentation/documentation purposes.

3.4 Technical Architecture

Concurrent Pipeline

Phase 3 runs three concurrent pipelines that must be precisely synchronized:

Pipeline	Rate	Actor/Thread	Output
AR Mesh Updates	60fps (ARSession delegate)	Main thread (ARKit requirement)	ARMeshAnchor additions/updates
Wi-Fi Measurement	2 Hz (every 500ms)	WiFiMeasurementEngine actor	MeasurementPoint with AR position tag
Map + Heatmap Render	10 Hz (every 100ms)	Metal render thread	Updated 2D map image with heatmap colors

Data Flow

53. **AR frame arrives** (60fps): Update mesh geometry. Extract camera transform (user position). Send position to render pipeline.
54. **Wi-Fi sample arrives** (2Hz): Tag with current AR position. Append to SurveyProject.measurementPoints. Notify render pipeline.
55. **Render tick** (10Hz): Project latest mesh to 2D. Rasterize new mesh triangles. Paint Wi-Fi color at recent measurement positions (circular splat with Gaussian falloff). Composite onto existing map texture (additive, not full re-render). Display in bottom-half view.

Performance Budget

Resource	Budget	Strategy
CPU	<60% sustained	Offload mesh projection to Metal compute. Wi-Fi polling is lightweight.
GPU	<40% sustained	Metal render of 2D map only (not full 3D scene). 10Hz update, not 60Hz.
Memory	<600MB total	Downsample mesh to 5cm resolution. Discard raw mesh after projection. Keep only 2D map texture + measurement points.
Battery	<15% per 30min scan	Reduce AR frame rate to 30fps during scan (sufficient for walking). Batch measurement writes.
Thermal	Stay below .serious	Monitor ProcessInfo.thermalState. If .serious: reduce mesh resolution, pause mesh updates, continue Wi-Fi sampling only. If .critical: auto-pause scan with user notification.
Storage	<50MB per scan session	Measurements are small (~200 bytes each). 2Hz x 30min = 3,600 points = ~720KB. Map texture is the largest item.

Metal Rendering Strategy

The 2D map + heatmap is rendered using Metal for performance. The rendering uses a persistent texture that is incrementally updated, not re-rendered from scratch each frame.

- **Map texture:** 2048x2048 RGBA texture representing the 2D floor plan. Initialized to transparent. Mesh triangles are projected and rasterized onto this texture as walls/floors.
- **Heatmap texture:** Separate 2048x2048 texture for signal colors. Wi-Fi measurements paint circular splats (Gaussian falloff, radius proportional to measurement spacing). Final display composites map + heatmap with configurable opacity.
- **Incremental updates:** Each render tick only processes NEW mesh anchors and NEW measurement points since the last tick. Previous results are preserved in the persistent textures.
- **Viewport:** The 2D map view auto-scrolls to keep the user's position centered, with pinch-to-zoom for reviewing previously scanned areas.

3.5 Wi-Fi Measurement During Continuous Scan

Continuous scanning produces dense measurement data (2 samples/second). This changes the interpolation strategy compared to Phase 1's sparse point measurements:

- **Real-time coloring:** During the scan, use nearest-neighbor coloring (paint a circle at each measurement point) rather than IDW interpolation. This is fast and gives immediate visual feedback.
- **Post-scan refinement:** When the user taps Finish, run full IDW interpolation over all points to produce a smooth, polished heatmap. This may take 2–5 seconds for a large scan (>3000 points) and should show a progress indicator.
- **Downsampling:** If the user walks slowly in one area, measurements cluster. Downsample to max 1 point per 0.5m² grid cell for interpolation (keep the median RSSI).
- **Walking speed detection:** Monitor AR camera velocity. If the user is stationary for >3 seconds, reduce measurement rate to 0.5Hz to avoid redundant data. Resume 2Hz when movement detected.

3.6 Acceptance Criteria

ID	Criterion
AC-3.1	Walking at normal pace (~1m/s), the 2D map renders new areas within 200ms of scanning them
AC-3.2	Heatmap colors appear on the map within 1 second of the user walking through an area
AC-3.3	30-minute continuous scan session uses <600MB memory and stays below thermal .serious
AC-3.4	Pause/resume preserves all data and AR tracking relocates within 5 seconds
AC-3.5	Post-scan IDW refinement completes in <5 seconds for a 2000-point scan
AC-3.6	Generated map walls align within 15cm of actual wall positions (LiDAR devices)
AC-3.7	Real-time heatmap colors match post-scan refined heatmap within 5dBm for 90% of points
AC-3.8	AP roaming boundaries (P1) are shown within 2m of actual roaming transition point
AC-3.9	Scan works in hallways, open offices, and rooms with furniture without manual intervention
AC-3.10	Battery consumption <15% for a 30-minute scan on iPhone 14 Pro

3.7 UI/UX Specification

Scanning View Layout

- **Top section (40%):** Live AR camera feed with detected surfaces highlighted. Floor surfaces show a semi-transparent green overlay. Walls show blue edge lines. The user's walking path shown as a thin white line.
- **Bottom section (60%):** 2D top-down map that grows as the user walks. Heatmap colors fill in behind the user. Current position: blue pulsing dot. Unmapped areas: dark grey. Pinch-to-zoom and drag to review.
- **Floating controls:** Pause/Resume button (top-left). Signal strength badge (top-right, shows current dBm). Point count + coverage % (bottom-left). Finish Scan button (bottom-right, prominent).
- **Transition to review:** After "Finish Scan," show full-screen map with heatmap overlay. Visualization type picker at top. Zoom/pan enabled. Share/Export button.

Color Scheme (WiFiman Convention)

Match WiFiman's color convention for user familiarity:

Signal Strength	Color	Hex	Label
-30 to -50 dBm	Blue/Cyan	#00BFFF → #00FF88	Excellent
-50 to -60 dBm	Green	#00FF88 → #88FF00	Good
-60 to -70 dBm	Yellow	#88FF00 → #FFFF00	Fair
-70 to -80 dBm	Orange	#FFFF00 → #FF8800	Weak
-80 to -90+ dBm	Red	#FF8800 → #FF0000	Dead Zone

3.8 Success Metrics

Metric	Target	Measurement
Continuous scan adoption	>50% of AR surveys use continuous mode over Phase 2 two-step	Analytics: surveyMode
Scan completion rate	>80% of started continuous scans reach Finish Scan	Analytics event tracking
Map coverage density	>90% of mapped floor area has at least 1 measurement point per 2m²	Post-scan analysis
Thermal throttle rate	<5% of scans trigger .serious thermal state	ProcessInfo monitoring
Real-time render FPS	>8 FPS sustained for 2D map updates during scan	Metal GPU profiler

3.9 Open Questions

Question	Owner	Status
Should the AR camera feed be optional (hide-able) to save battery?	Design	Open
Metal vs SceneKit for the 2D map rendering?	Engineering	Metal recommended for incremental updates
How to handle outdoor areas (AR tracking degrades in sunlight)?	Engineering	Open — propose indoor-only for v1
Should we support iPhone SE (no LiDAR) with degraded continuous scan?	Blake	Open — recommend LiDAR-only for Phase 3
Maximum recommended scan area per session?	Engineering	Needs testing — estimate ~500m² within memory budget
Can we run a lightweight speed test during continuous scan without stalling the UI?	Engineering	Open — may need to limit to passive RSSI only during walk, active test at pinned points

Implementation Dependencies & Timeline

Dependency Chain

Component	Depends On	Required By
HeatmapSurveyModels (Core)	Nothing	All phases
WiFiMeasurementEngine (Core)	Existing WiFiInfoService, PingService, SpeedTestService	All phases
HeatmapRenderer / IDW (Core)	HeatmapSurveyModels	All phases
macOS Blueprint UI	Core models + renderer	Phase 1
iOS Blueprint UI	Core models + renderer	Phase 1
.netmonsurvey file format	Core models	Phase 1 (save/load)
AR Floor Plan Generator	Core models, ARKit, Metal	Phase 2, Phase 3
AR Position Tracker	AR Floor Plan Generator	Phase 2, Phase 3
Continuous Measurement Stream	WiFiMeasurementEngine	Phase 3
Metal Incremental Map Renderer	AR Floor Plan Generator, HeatmapRenderer	Phase 3

Recommended Build Order

56. **Shared foundation (1–2 weeks):** Build HeatmapSurveyModels, WiFiMeasurementEngine, HeatmapRenderer in NetMonitorCore. Write unit tests for IDW interpolation and data model serialization.
57. **Phase 1 macOS (2–3 weeks):** Full macOS blueprint survey UI. This is the most feature-rich implementation and validates the entire data model.
58. **Phase 1 iOS (1–2 weeks):** iOS blueprint survey UI. Reuses all core logic, adds touch-optimized UI with liquid glass styling.
59. **Phase 2 iOS (2–3 weeks):** AR floor plan generation + survey. Builds on Phase 1 iOS survey UI, adds ARKit integration.
60. **Phase 3 iOS (3–4 weeks):** Continuous scan. Most complex phase. Requires Metal rendering pipeline and careful performance optimization.

Estimated Total: 9–14 weeks

Each phase is independently shippable. Phase 1 alone delivers significant value and validates the architecture for Phases 2 and 3.

Risk Assessment

Risk	Likelihood	Impact	Mitigation
iOS Wi-Fi API restrictions tighten in future iOS versions	Medium	High	Abstract all Wi-Fi access behind WiFiMeasurementEngine protocol. If APIs change, only one implementation file updates.
AR mesh-to-2D floor plan produces noisy results	Medium	Medium	Phase 2 is P1 nice-to-have. Fallback to manual floor plan import always available. RoomPlan API (P2) produces cleaner output.
Continuous scan (Phase 3) thermal throttling on older devices	High	Medium	ThermalThrottleMonitor (already exists in NetworkScanKit) gates scan intensity. Auto-pause at .serious. Document minimum device: iPhone 12 Pro.
Metal rendering pipeline complexity	Medium	High	Phase 3 Metal renderer is the most complex new code. Prototype early. Fallback: CoreGraphics-based renderer at lower frame rate.
NEHotspotNetwork rate limiting by Apple	Low	High	Cache measurements for 1s minimum. If throttled, reduce polling rate and notify user.

Appendix: Reference App Comparison

Feature	NetSpot (macOS)	WiFiman (iOS)	NetMonitor Target
Blueprint import	Yes — PNG, PDF, CAD	No (LiDAR only)	Yes — Phase 1
Draw floor plan	Yes — built-in editor	No	Phase 1 P1 (macOS)
Walk survey (tap to measure)	Yes — primary workflow	No	Yes — Phase 1
AR floor plan generation	No	Yes — LiDAR Floorplan Mapper	Phase 2
AR continuous scan	No	Yes — Signal Mapper	Phase 3
Heatmap visualizations	20+ types	Signal strength only	5–8 types (phased)
Active scan (speed test at point)	Yes	No	Phase 1 P1
Predictive survey	Yes (Pro tier)	No	Not planned (future)
Vendor lock-in	None	Best with UniFi console	None
Price	$49–$899	Free	Included in NetMonitor
Export	PDF, CSV	Limited	PDF, .netmonsurvey

Sources: *netspotapp.com*, *help.ui.com (WiFiman)*, *community.ui.com*, *larsklint.com*